OOPs(Object Oriented Programming System)

26 August 2025 19:52

Difference between loops,Functions,OOPs:-

Feature / Concept	Loops	Functions	OOP (Object-Oriented Programming)
Definition	Used to repeat a block of code multiple times.	A block of code that performs a specific task and can be reused.	A programming paradigm based on objects and classes.
Purpose	To execute repetitive tasks efficiently.	To organize code into reusable, modular blocks.	To model real-world entities and relationships using classes & objects.
Code Reusability	Limited – repeats the same block but not reusable across programs.	High – functions can be reused multiple times.	Very High – classes, inheritance, and polymorphism allow reusing code without rewriting.
Code Redundancy	High – same logic may be repeated in multiple loops.	Moderate – reduces redundancy to some extent.	Very Low – avoids duplication through inheritance and class structures.
Proper Structure	No formal structure, just repetition.	Some structure – organizes tasks into blocks.	Strong structure – organizes code into classes & objects, making it easy to maintain.
Code Flexibility	Low – changes require editing loops everywhere.	Medium – changes can be handled by modifying the function.	High – flexible, scalable, and easy to extend without breaking existing code.
Collaboration in Projects	Minimal – loops are simple, not suitable for team-based modular work.	Good for small modules, but difficult for large teams.	Excellent – OOP supports modular design, so multiple developers can work on different classes simultaneously.
Level of Abstraction	Low – direct repetition of tasks.	Medium – abstracts tasks into callable blocks.	High – abstracts both data and behavior into reusable objects.
Real-Life Example	Repeating "Take attendance" for every student in a class.	A calculator function that adds numbers whenever called.	A Student class with attributes (name, roll_no) and methods (attend_class()).
When to Use	When tasks are repetitive.	When you need to organize code into small reusable units.	When you want to design large, scalable, real-world systems.

Class:

- A **class** is a **blueprint or template** used to create **objects**.
- It groups together **data (variables/attributes)** and **functions (methods)** in one place.
- Example: A *Car* class can have attributes like color, brand and methods like start() or stop().

(OR)

Class is basically a container which is used to store the data we can utilize the store data.

(OR)

Class is a container which is used to store the properties and functionalities of a real time entity.

(OR)

Class is a blue print which consist of properties and functions of object .

Creation of class:

```
class ClassName:
   # attributes (variables)
   # methods (functions)
```

Example:
```
class Student:
    attributes (variables)
     # methods (functions)
```

Q.How can we access Class members?

Syntax:- classname.Property or attrite or method name

Q. What is the difference between a method and a normal function?

- **Function** → independent block of code.
- **Method** → function defined inside a class, works with objects.

Q. Why do we need a class if we can write code directly using functions and variables?

- Functions handle small tasks, but classes combine **data + behavior**.
- For large projects, classes give **structure, reusability, and scalability.**

Q. Is a class compulsory in Python programming?

- No , small programs can be written without classes.
- But for **OOP, modularity, and real-world projects**, classes are essential.

Q. Can we create a class without attributes or methods?

- Yes , but it's usually not useful. Example:

```
Ex:class Empty:
   pass
```

Q. What are the Rules of Class?

Rules of Class :-

1. Defining a Class

- ✧ A class is created using the **class keyword**.
- ✧ The **class name** should follow Python naming rules (PascalCase is recommended).

```
Ex:class Student:
   pass
```

2. Class Name Rules

- ✧ Must start with a **letter** or **underscore (_)**.
- ✧ Cannot start with a number.
- ✧ Should not be a Python **keyword** (e.g., class, def, for).
- ✧ By convention, **use CapitalizedWords (PascalCase)**.

  Example: CarModel, EmployeeDetails.

3. Attributes and Methods

- ✧ Inside a class, we can define:
  - **Variables** → Called *attributes* or *data members*.
  - **Functions** → Called *methods*.
- ✧ Attributes and methods belong to objects created from the class.

4. Indentation Rule

- ✧ All class members (variables, methods) must follow **proper indentation** under the class block.

Q.Can we Create a User defined Classes or Data types?

YES,We can …

- ✧ **Built-in Data Types** → Already defined classes by Python (int, str, list …).
- ✧ **User-Defined Data Types** → Classes that we create (Student, Car, Employee).

- ✧ In Python, **data types = classes**.
- ✧ Your variables are just **objects of those classes**.

- Example:
  - 10 → object of class int
  - "Hello" → object of class str
  - [1,2,3] → object of class list

Object:

- An **object** is an **instance of a class**.
- It is a **real-world entity** created from the **blueprint (class)**.
- Objects hold **data (attributes)** and can perform **actions (methods).**

Creation of Object:

- **Syntax**: object_name = ClassName(arguments)

Accessing Object members:

Syntax:object_name.object_member_name

Note:**Object members** are instance variables and methods defined inside a class. They are accessed using the dot (.) operator on the object

Q. How many objects can we create from a class?

- Unlimited . Every object will have its own copy of instance variables.

Q. Where is object data stored in memory?

- In the **heap memory**, while class definition itself is stored in memory space for classes.

Q. What happens if we delete an object?

- Python's **garbage collector** frees memory automatically.

  What is Garbage Collection?

  - When we create objects in Python, they use **memory (RAM)**.
  - If an object is **no longer needed** (no variable is referring to it), it becomes **garbage**.
  - Python has a built-in process called **Garbage Collector (GC)** that **automatically frees memory** used by such unused objects.

Programs On Class and Object:

Example:1

```
class Bank:
   bank_name = 'SBI'       # class variable
   bank_location = 'Mysore'  # class variable

c1 = Bank()   # Object 1
c2 = Bank()   # Object 2

# ---- Before modification ----
print(Bank.bank_name, Bank.bank_location)   # → SBI Mysore
print(c1.bank_name, c1.bank_location)       # → SBI Mysore (reads from class)
print(c2.bank_name, c2.bank_location)       # → SBI Mysore (reads from class)

# ---- Modifications ----
Bank.bank_location = 'Banglore'          # change CLASS variable
 c1.bank_location = 'Hyd'               # change object bank location

# ---- After modification ----
print(Bank.bank_name, Bank.bank_location) #-->SBI Banglore
print(c1.bank_name, c1.bank_location)       # → SBI Hyd  (name from class, location from c1)
print(c2.bank_name, c2.bank_location)       # → SBI Banglore (still reading class value)
```

Output:

Before modification:

```
SBI Mysore    # c1 reads from class
SBI Mysore    # c2 reads from class
SBI Mysore    # direct class access
```

After modification

SBI Banglore # class variable changed
SBI Hyd # c1 has its own instance variable now
SBI Banglore # c2 still reads from updated class variable

Note :

Class variables (data shared by all objects)

- bank_name, bank_location defined inside the class body are **class variables**.
- Reading them via either the class (Bank.bank_name) **or** an object (c1.bank_name) works if the object doesn't have an instance variable with the same name.

Changing class variables vs. instance variables

- Bank.bank_location = 'Banglore' **updates the shared class value** → affects all objects that haven't overridden it.
- c1.bank_location = 'Hyd' **does not change the class**; it **creates/overrides an instance variable on c1 only** (shadowing the class variable)

Modification with respect to Class	Modification with respect to Object
❖ Class will get effected along with that all the objects.	❖ It is going to effected only that object it keep remaining as it is.

Q. Can we modify object (instance) data using the class name?

No,

- Because class doesn't know individual object's data.
- Instance variables belong only to that specific object.

Q. Can we modify class data using the object?

Yes,But directly writing obj.var = value creates a **new instance variable**, it won't change class data.

Example 2:

```
class School:
    school_name = 'DPS'
    city = Hyderabad'

s1 = School()
s2 = School()

print(s1.school_name, s1.city)
print(s2.school_name, s2.city)

# Modify class variable
School.city = "Delhi"
print(s1.school_name, s1.city)
print(s2.school_name, s2.city)

# Modify object variable
s1.city = "Chennai"
print(s1.school_name, s1.city)
print(s2.school_name, s2.city)
```

Practice Questions on Class & Object :

1.Create class called College with class members and objects.

Example:-

- Create a class College with class variables college_name and location.
- Create multiple objects for students and print their college details.
- Try modifying college location using the class name and using the object

2.Create class called Car with class members and objects.

Example:

- Create a class Car with variables brand, model, and price.
- Create two car objects and show the difference if you change price using the class and using the object.

3.Create class called Hospital with class members and objects

4.Create class called Library with class members and objects
5.Create class called Mobile with class members and objects
6.Create class called Company with class members and objects

Note: For the above practice questions, you can **choose your own class variables** as per your imagination.